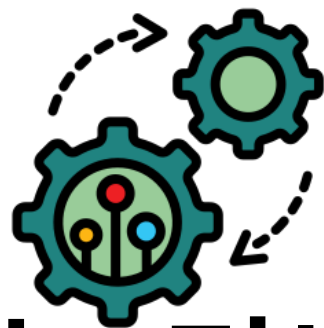




케이녹 리버싱 3회차 과제

17기 최동현 (202511688)

1-1. Stack Frame Architecture

• 스택 프레임 (Stack Frame)

： 스택 프레임(Stack Frame)은 함수가 호출될 때 해당 함수의 실행 컨텍스트(상태)를 유지하기 위해 프로세서의 스택 세그먼트에 할당되는 독립적인 메모리 블록입니다. x86/x64 아키텍처에서 스택은 높은 주소에서 낮은 주소로 성장하며, 후입선출(LIFO) 구조를 가집니다.

** 스택 프레임 데이터 구성*

- **Local Variables (지역 변수)** : 함수 내부에서 선언된 변수들이 할당되는 공간입니다.
- **SFP (Saved Frame Pointer)** : 함수가 종료된 후, 호출한 함수(Caller)의 스택 프레임 기준점(Base Pointer)으로 되돌아가기 위해 백업해 둔 호출자의 RBP 레지스터 값입니다.
- **RET (Return Address)** : 함수 실행이 완료된 후 CPU의 명령어 포인터(RIP)가 돌아가야 할 **다음 실행 명령어의 메모리 주소**입니다.

1-2. Stack Frame Architecture

- **호출 시 동작 (Call):** call 명령어가 실행되면 CPU는 내부적으로 현재 RIP를 스택에 push하여 RET 주소를 백업하고, 대상 함수의 주소로 jmp합니다.
- **종료 시 동작 (Return):** ret 명령어가 실행되면 CPU는 스택 최상단에 있는 RET 값을 pop하여 RIP 레지스터에 대입함으로써 실행 흐름을 복귀시킵니다.

```
Cyber@DESKTOP-CYBER > Shared >
gdb -q .\test.exe
Reading symbols from .\test.exe...
(gdb) break func
Breakpoint 1 at 0x140001454
(gdb) run
Starting program: C:\Shared\test.exe
[New Thread 19192.0x712c]

Thread 1 hit Breakpoint 1, 0x00007ff70fa11454 in func ()
(gdb) info frame
Stack level 0, frame at 0x5ffe20: 스택 프레임 기준 주소
rip = 0x7ff70fa11454 in func; saved rip = 0x7ff70fa11480
called by frame at 0x5ffe60
Arglist at 0x5ffe10, args:
Locals at 0x5ffe10, Previous frame's sp is 0x5ffe20
Saved registers:
rbp at 0x5ffe10, rip at 0x5ffe18
```

현재 rip

함수 실행 완료 후 돌아갈 위치

2. Function Prologue & Epilogue

- **함수 프롤로그 (Function Prologue)** : 스택 프레임을 생성하는 과정

** 함수 프롤로그*

1. `push rbp` : 호출자(Caller)의 RBP 값을 스택에 백업한다. (이 부분이 SFP가 된다)
2. `mov rbp, rsp` : 현재 스택 포인터(RSP)의 주소를 RBP에 대입하여 새로운 스택 프레임의 기준점을 설정한다.
3. `sub rsp, 0x10` : 스택 포인터를 감소시켜 지역 변수가 사용할 공간을 할당한다.

- **함수 에필로그 (Function Epilogue)** : 스택 프레임을 해제하고 복귀하는 과정

** 함수 에필로그*

1. `leave: mov rsp, rbp` 지역 변수 공간 해제 후 `pop rbp` (호출자의 RBP 복구)를 수행한다.
2. `ret`: `pop rip`를 내부적으로 수행, 스택 최상단에 있는 `ret` 주소로 점프한다.

3. 함수 호출 규약

- **cdecl (C Declaration)**

： 32비트(x86) C 프로그램의 기본 규약입니다. 인자를 **스택 오른쪽에서 왼쪽 순서로 Push**하여 전달합니다. 함수 종료 후 스택의 정리는 호출자(Caller)가 수행합니다.

- **stdcall**

： 32비트 Windows API의 기본 규약입니다. 인자를 스택에 푸시하는 것은 같으나, 스택 정리는 피호출자(Callee)가 `ret n` 명령어를 통해 수행합니다.

- **fastcall (System V AMD64 ABI)**

： 64비트(x86-64) 리눅스 환경의 표준입니다. 메모리 접근(스택)은 느리므로, **처음 6개의 인자는 레지스터를 사용하여 전달**하고, 7번째 인자부터 스택을 사용합니다.

* **인자 전달 순서**: `RDI → RSI → RDX → RCX → R8 → R9` 순서로 레지스터에 값을 넣은 뒤 `call` 명령을 수행합니다.

4. Dreamhack (rev-basic-0)

```
IDA View-A Pseudocode-A
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_140001190("Input : ", argv, envp);
7     sub_1400011F0("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

```
Cyber@DESKTOP-CYBER Downloads
> .\chall0.exe
Input : Compar3_the_string
Correct
```

```
IDA View-A Pseudocode-A
1 BOOL __fastcall sub_140001000(const char *a1)
2 {
3     return strcmp(a1, "Compar3_the_string") == 0;
4 }
```

4. Dreamhack (rev-basic-1)

```
IDA View-A Pseudocode-A
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_140013E0("Input : ", argv, envp);
7     sub_14001440("%256s", v4);
8     if ( (unsigned int)sub_14001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

```
chall1.py
1 value = {
2     chr(67) + chr(111) + chr(109) + chr(112) + chr(97) + chr(114) + chr(51) +
3     chr(95) + chr(116) + chr(104) + chr(101) + chr(95) + chr(99) + chr(104) +
4     chr(52) + chr(114) + chr(97) + chr(99) + chr(116) + chr(51) + chr(114)
5 }
6
7 print(value)
```

```
Cyber@DESKTOP-CYBER Shared
→ python .\chall1.py
{'Compar3_the_ch4ract3r'}
```

```
Cyber@DESKTOP-CYBER Downloads
→ .\chall1.exe
Input : Compar3_the_ch4ract3r
Correct
```

```
IDA View-A Pseudocode-A
1 BOOL __fastcall sub_14001000(_BYTE *a1)
2 {
3     if ( *a1 != 67 )
4         return 0LL;
5     if ( a1[1] != 111 )
6         return 0LL;
7     if ( a1[2] != 109 )
8         return 0LL;
9     if ( a1[3] != 112 )
10        return 0LL;
11    if ( a1[4] != 97 )
12        return 0LL;
13    if ( a1[5] != 114 )
14        return 0LL;
15    if ( a1[6] != 51 )
16        return 0LL;
17    if ( a1[7] != 95 )
18        return 0LL;
19    if ( a1[8] != 116 )
20        return 0LL;
21    if ( a1[9] != 104 )
22        return 0LL;
23    if ( a1[10] != 101 )
24        return 0LL;
25    if ( a1[11] != 95 )
26        return 0LL;
27    if ( a1[12] != 99 )
28        return 0LL;
29    if ( a1[13] != 104 )
30        return 0LL;
31    if ( a1[14] != 52 )
32        return 0LL;
33    if ( a1[15] != 114 )
34        return 0LL;
35    if ( a1[16] != 97 )
36        return 0LL;
37    if ( a1[17] != 99 )
38        return 0LL;
39    if ( a1[18] != 116 )
40        return 0LL;
41    if ( a1[19] != 51 )
42        return 0LL;
43    if ( a1[20] == 114 )
44        return a1[21] == 0;
45    return 0LL;
46 }
```

4. Dreamhack (rev-basic-2)

```
IDA View-A Pseudocode-A
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011B0("Input : ", argv, envp);
7     sub_140001210("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

.data:0000000140003000 aC db 'C',0 ; DATA XREF: sub_140001000+281o
.data:0000000140003002 align 4
.data:0000000140003004 aO db 'o',0
.data:0000000140003006 align 8
.data:0000000140003008 aM db 'm',0
.data:000000014000300A align 4
.data:000000014000300C aP db 'p',0
.data:000000014000300E align 10h
.data:0000000140003010 a4 db '4',0
.data:0000000140003012 align 4
.data:0000000140003014 aR db 'r',0
.data:0000000140003016 align 8
.data:0000000140003018 aE db 'e',0
.data:000000014000301A align 4
.data:000000014000301C db '_',0
.data:000000014000301E align 20h
.data:0000000140003020 aT db 't',0
.data:0000000140003022 align 4
.data:0000000140003024 db 'h',0
.data:0000000140003026 align 8
.data:0000000140003028 aE_0 db 'e',0
.data:000000014000302A align 4
.data:000000014000302C db '_',0
.data:000000014000302E align 10h
.data:0000000140003030 aA db 'a',0
.data:0000000140003032 align 4
.data:0000000140003034 aR_0 db 'r',0
.data:0000000140003036 align 8
.data:0000000140003038 aR_1 db 'r',0
.data:000000014000303A align 4
.data:000000014000303C a4_0 db '4',0
.data:000000014000303E align 20h
.data:0000000140003040 aY db 'y',0
.data:0000000140003042 align 10h

```
IDA View-A Pseudocode-A
1 int64 __fastcall sub_140001000(int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned int64)i < 0x12; ++i )
6     {
7         if ( *(_DWORD *)&aC[4 * i] != *(unsigned __int8 *)(a1 + i) )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

```
Cyber@DESKTOP-CYBER Downloads
> .\chall2.exe
Input : Comp4re_the_arr4y
Correct
```

5. xor_plus

```
CH0I@DESKTOP-CH0I  xor_plus-1
→ .\xor_plus.exe
usage: C:\Users\CH0I\Downloads\xor_plus-1\xor_plus.exe <key> <flag>
press any key to exit....
```

“xor_plus” <key> <flag> 방식 -> int main(int argc, char** argv) 방식

0040147F	mov dword ptr ss:[esp],xor_plus.405076	00405076	"usage: %s <key> <flag>\n"
00401488	mov dword ptr ss:[esp],xor_plus.40508E	0040508E	"press any key to exit.... "
004014AE	mov dword ptr ss:[esp+4],xor_plus.4050AB	004050AB	"K.knock2026"
004014C2	mov dword ptr ss:[esp],xor_plus.4050B7	004050B7	"wrong key"
004014CE	mov dword ptr ss:[esp],xor_plus.40508E	0040508E	"press any key to exit.... "
004014FE	mov dword ptr ss:[esp],xor_plus.4050C1	004050C1	"wrong length"
0040150A	mov dword ptr ss:[esp],xor_plus.40508E	0040508E	"press any key to exit.... "
0040158D	mov dword ptr ss:[esp],xor_plus.4050CE	004050CE	"nope"
00401599	mov dword ptr ss:[esp],xor_plus.40508E	0040508E	"press any key to exit.... "
004015C1	mov dword ptr ss:[esp],xor_plus.4050D3	004050D3	"correct!"
004015D9	mov dword ptr ss:[esp],xor_plus.4050DC	004050DC	"flag: %s\n"

00401470	837D 08 03	cmp dword ptr ss:[ebp+8],3	
00401474	74 30	je xor_plus.4014A6	
00401476	8B45 0C	mov eax,dword ptr ss:[ebp+C]	
00401479	8B00	mov eax,dword ptr ds:[eax]	
0040147B	894424 04	mov dword ptr ss:[esp+4],eax	
0040147F	C70424 76504000	mov dword ptr ss:[esp],xor_plus.405076	[esp]:RtlIsCurrentThreadAttachExempt+1259, 405076:"usage: %
00401486	E8 55270000	call <JMP.&printf>	
00401488	C70424 8E504000	mov dword ptr ss:[esp],xor_plus.40508E	[esp]:RtlIsCurrentThreadAttachExempt+1259, 40508E:"press ar
00401492	E8 49270000	call <JMP.&printf>	
00401497	E8 64270000	call <JMP.&getchar>	
0040149C	B8 01000000	mov eax,1	
004014A1	E9 49010000	jmp xor_plus.4015EF	
004014A6	8B45 0C	mov eax,dword ptr ss:[ebp+C]	

통상적으로 [ebp+8] = argc, [ebp+C] = argv 이므로 if (argc == 3) { eax = argv } 로 처음 안내 멘트를 넘어감
argv[0] = "xor_plus", argv[1] = "key", argv[2] = "flag" 이므로 총 카운트가 3이어야함

5. xor_plus

004014A6	8B45 0C	mov eax,dword ptr ss:[ebp+C]	
004014A9	83C0 04	add eax,4	
004014AC	8B00	mov eax,dword ptr ds:[eax]	
004014AE	C74424 04 AB504000	mov dword ptr ss:[esp+4],xor_plus.4050AB	4050AB:"K.knock2026"
004014B6	890424	mov dword ptr ss:[esp],eax	[esp]:RtlIsCurrentThread
004014B9	E8 FA260000	call <JMP.&strcmp>	
004014BE	85C0	test eax,eax	
004014C0	74 27	je xor_plus.4014E9	

EAX = argv 값을 넣고

EAX 주소의 + 4를 하게 됨 (포인터 하나 4바이트) -> argv[1]

mov eax,dword ptr ds:[eax]를 통해 실제 argv[1]이 가리키는 값, 즉 <key>값을 eax에 넣음 EAX = argv[1];

그 다음 주소 4050AB에 있는 문자열 값 <"K.knock2026">을 esp+4에 가져옵니다. 밑에 보면 strcmp함수로 콜하는게 보이는데 제 예상 strcmp()함수의 파라미터 strcmp(esp+0, esp+4) 이렇게 들어가는듯합니다 (포인터 4바이트)

아까 eax = argv[1]값을 넣었던 eax를 mov [esp], eax로 esp에 넣은 다음에 strcmp로 call해서 strcmp(argv[1], "K.knock2026")이 되게 됩니다.

밑에 test eax,eax는 eax가 0인가를 확인하는 opcode로 strcmp()함수 자체가 같으면 0을 반환하기에 문자열이 같은지를 확인하고 같다면 je 4014E9로 if(!strcmp(argv[1], "K.knock2026")) { 4014E9코드 } 가 되지않을까 싶습니다.

5. xor_plus

004014E9	8B45 0C	mov eax,dword ptr ss:[ebp+C]
004014EC	83C0 08	add eax,8
004014EF	8B00	mov eax,dword ptr ds:[eax]
004014F1	890424	mov dword ptr ss:[esp],eax
004014F4	E8 AF260000	call <JMP.&strlen>
004014F9	83F8 12	cmp eax,12
004014FC	74 27	je xor_plus.401525

사진처럼 wrong key와는 달리 wrong length가 나왔고 key는 맞춘것을 확인

넘어온 4014E9 주소부터 코드를 읽어보면 위에서 했던 방식과 같음을 알 수 있음

Eax에 [EBP+C]를 넣음 => eax = argv

Eax주소에 8을 더함 => argv[2]

Argv[2]가 가리키는 값을 eax에 넣음 => eax = argv[2];

Mov [esp], eax통해 스택에 넣고 call strlen로 strlen함수를 호출

밑에 cmp eax,12가 있고 je 주소값이 있는것으로 보아 if문인걸 확인

if(strlen(argv[2]) == 0x12) { 401525코드 } 라고 유추해볼 수 있음. 0x12의 10진수값은 18임 $[(16 * 1) + (1 * 2) = 18]$

따라서 플래그값의 길이는 18자리라는것을 알수있다.

```
CH0I@DESKTOP-CH0I  xor_plus-1
> .\xor_plus.exe "K.knock2025" "test"
wrong key
press any key to exit....
CH0I@DESKTOP-CH0I  xor_plus-1
> .\xor_plus.exe "K.knock2026" "test"
wrong length
press any key to exit....
CH0I@DESKTOP-CH0I  xor_plus-1
```

5. xor_plus

```
--> 00401525 C74424 1C 00000000 mov dword ptr ss:[esp+1C],0
--> 0040152D E9 84000000 jmp xor_plus.4015B6
```

```
CH0I@DESKTOP-CH0I xor_plus-1
-> .\xor_plus.exe "K.knock2026" "1234567890123456789"
wrong length
press any key to exit....
CH0I@DESKTOP-CH0I xor_plus-1
-> .\xor_plus.exe "K.knock2026" "123456789012345678"
nope
press any key to exit....
```

사진처럼 wrong length와는 달리 nope가 나왔고 key의 길이까지 맞춘것을 확인.

넘어온 40152D 주소부터 코드를 다시 읽어보면, esp+1C의 어느 변수에 0 값을 넣고 4015B6으로 점프하는것을 확인

```
004015B6 837C24 1C 11 cmp dword ptr ss:[esp+1C],11
004015B8 0F86 71FFFFFF jbe xor_plus.401532
```

보면 아까 변수와 0x11값인 17을 비교함. 그 다음 jbe면 작거나 같을때 ESP+1C <= 17면 401532주소로 간다.

```
00401532 8B45 0C mov eax,dword ptr ss:[ebp+C]
00401535 83C0 08 add eax,8
00401538 8B10 mov edx,dword ptr ds:[eax]
0040153A 8B4424 1C mov eax,dword ptr ss:[esp+1C]
0040153E 01D0 add eax,edx
00401540 0FB600 movzx eax,byte ptr ds:[eax]
00401543 89C6 mov esi,eax
```

그 다음 401532로 와서 보면 eax = argv, eax+8, ... 결론적으로 edx = argv[2]값인 flag를 edx넣는다.

다음 아까 변수 esp+1C = 0값을 eax에 넣는다. EAX = esp+1C변수 그리고 add eax,edx로 플래그 시작주소에서 esp+1C값을 추가한다. 따라서 위에서부터 예상해보면 esp+1C은 i로 알 수 있고 반복문을 돌리는 형식으로 볼 수 있다.

즉 for(i = 0; i <= 17; i++) { }가 되고 movzx eax, ds:[eax]로 EAX = flag[i];볼 수 있다.

마지막 mov esi,eax로 현재 flag[i]값을 esi인 출발지 인덱스에 보관하게 된다.

5. xor_plus

00401545	8B45 0C	mov eax,dword ptr ss:[ebp+C]
00401548	83C0 04	add eax,4
0040154B	8B18	mov ebx,dword ptr ds:[eax]

그 다음으로 이번엔 `eax = argv[1];` 인 `key`값의 시작주소를 `ebx`에 저장한다.

0040154D	8B4C24 1C	mov ecx,dword ptr ss:[esp+1C]
00401551	BA A38B2EBA	mov edx,BA2E8BA3
00401556	89C8	mov eax,ecx
00401558	F7E2	mul edx
0040155A	C1EA 03	shr edx,3
0040155D	89D0	mov eax,edx
0040155F	C1E0 02	shl eax,2
00401562	01D0	add eax,edx
00401564	01C0	add eax,eax
00401566	01D0	add eax,edx
00401568	29C1	sub ecx,eax
0040156A	89CA	mov edx,ecx

`Ecx`에 `i`값을 넣고 `edx`엔 `BA2E8BA3` 을 넣고 `eax`에 `i`값 넣고 `eax`와 `edx`를 곱하여 결과값 `eax,edx`에 저장후 `shr`로 오른쪽으로 3칸 밀어버리고 `eax`에 `edx`값을 넣고 `eax`에 왼쪽으로 2칸 시프트하고 밀의 계산을 쫓합니다.

그래서 정확히 위의 뜻이 뭐냐하면 $i \% 11$ 를 계산하는 것으로 알 수 있습니다.

$i / 11$ 의 몫을 계산하고 몫 $\times 11$ 을 만든 후 `sub ecx, eax`해서 $i - (i / 11 * 11)$ 을 만들게 되고 이말은 즉 $i \% 11$ 를 의미하게 됩니다. 그래서 마지막 `mov edx,ecx`가 끝나면 `edx`에 $i \% 11$ 한 값이 들어가고 `EDX = i \% 11;`가 됩니다.

결론으로 $i = 0 \rightarrow 0, 1 \rightarrow 1, 10 \rightarrow 10, 11 \rightarrow 0$ 이렇게 `key` 11글자를 반복해서 사용하게 됩니다.

5. xor_plus

0040156C	8D0413	lea eax,dword ptr ds:[ebx+edx]
0040156F	0FB600	movzx eax,byte ptr ds:[eax]

Ebx = key 시작주소, edx = i % 11 이므로

Eax = &key[i % 11] 가 된다.

Movzx eax, byte ptr [eax]를 통해 eax = key[i % 11] 실제 문자를 하나 읽는다.

00401572	31F0	xor eax,esi
00401574	83C0 01	add eax,1
00401577	884424 1B	mov byte ptr ss:[esp+1B],al

Esi = flag[i]므로 eax = key[i % 11] XOR flag[i]가 된다. 그 후 add eax,1로 1을 더하게 되면

eax = (key[i % 11] ^ flag[i]) + 1이 된다. Mov [esp+1B],al을 통해 계산한 결과의 하위 1바이트를 임시공간에 저장합니다.

0040157B	8B4424 1C	mov eax,dword ptr ss:[esp+1C]
0040157F	05 64504000	add eax,xor_plus.405064
00401584	0FB600	movzx eax,byte ptr ds:[eax]
00401587	3A4424 1B	cmp al,byte ptr ss:[esp+1B]
0040158B	74 24	je xor_plus.4015B1

Eax = i, 405064와 더하게 되면 405064에 있는 배열 target[i]가 된다. Movzx eax,ds:[eax]를 통해 eax = target[i] 값을 넣고 밑의 cmp al,esp+1B를 통해 target[i]와 eax의 XOR식을 비교하여 검사를 하게 된다.

Je 4015B1를 통해 같다면 해당 주소로 이동하게되며 같지 않다면 mov esp, "nope" call puts로 nope를 출력하게 된다.

5. xor_plus

```
004015B1 834424 1C 01 add dword ptr ss:[esp+1C],1
004015B6 837C24 1C 11 cmp dword ptr ss:[esp+1C],11
004015BB 0F86 71FFFFFF jbe xor_plus.401532
```

i += 1하며 i <= 17을 비교해 작거나 같다면 다시 위 루프를 돌게 된다. 만약 i가 18이 되어 루프를 넘어가게 되면

```
004015C1 C70424 D3504000 mov dword ptr ss:[esp],xor_plus.4050D3 [esp]:RtlIsCurrentThreadAttachExempt+1259, 4050D3:"correct!"
004015C8 E8 0B260000 call <JMP.&puts>
004015CD 8B45 0C mov eax,dword ptr ss:[ebp+C]
004015D0 83C0 08 add eax,8
004015D3 8B00 mov eax,dword ptr ds:[eax]
004015D5 894424 04 mov dword ptr ss:[esp+4],eax
004015D9 C70424 DC504000 mov dword ptr ss:[esp],xor_plus.4050DC [esp]:RtlIsCurrentThreadAttachExempt+1259, 4050DC:"flag: %s\
004015E0 E8 FB250000 call <JMP.&printf>
004015E5 E8 16260000 call <JMP.&getchar>
```

Correct! 출력 후 eax =argv[2] 값을 flag: 와 함께 출력하고 프로그램이 끝나게 된다.

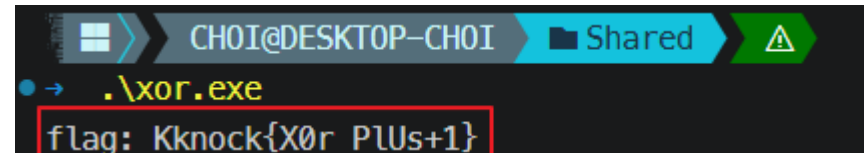
5. xor_plus

00405064	01 46 06 02	0D 09 11 68	01 41 6A 1C	43 3F 1E 45	.F.....k.Aj.C?.E
00405074	53 17 75 73	61 67 65 3A	20 25 73 20	3C 6B 65 79	S.usage: %s <key

target[] 해별의 주소인 0x405064 값을들을 가져와서

$\text{Target}[i] == (\text{key}[i \% 11] \wedge \text{flag}[i]) + 1$ 를 역산해서 계산하게되면 이와 같이 플래그가 나온다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     char flag[19];
5     const char key[12] = "K.knock2026";
6     const int target[19] = {
7         0x01, 0x46, 0x06, 0x02, 0x0D, 0x09,
8         0x11, 0x6B, 0x01, 0x41, 0x6A, 0x1C,
9         0x43, 0x3F, 0x1E, 0x45, 0x53, 0x17
10    };
11
12    for(int i = 0; i < 18; i++) {
13        flag[i] = (char)((target[i] - 1) ^ key[i % 11]);
14    }
15
16    flag[19] = '\0';
17    printf("flag: %s\n", flag);
18
19    return 0;
20 }
```



```
CHOI@DESKTOP-CHOI Shared
→ .\xor.exe
flag: Kknock{X0r_PlUs+1}
```



감사합니다

17기 최동현